

LEARNING TO THEORIZE IN MENTAL TERMS

MSc Thesis (*Afstudeerscriptie*)

written by

Gidon Kaminer

under the supervision of **Dr Fausto Carcassi**, and submitted to the
Examinations Board in partial fulfilment of the requirements for the degree of

MSc in Logic

at the *Universiteit van Amsterdam*.

Date of the public defence: **Members of the Thesis Committee:**

August 28th, 2026

Dr Fausto Carcassi (supervisor)

Dr Malvin Gattinger (chair)

Dr Martha Lewis

Dr Giorgio Sbardolini



INSTITUTE FOR LOGIC, LANGUAGE AND COMPUTATION

Abstract

ABSTRACT OF THE THESIS

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius.

Contents

1 Introduction: Theory of Mind	3
1.1 The nature and scope of theory of mind	3
1.1.1 the problem of other minds:	3
1.1.2 the function of ToM:	3
1.2 Developmental milestones (ToM scale)	4
1.2.1 early roots (0-12 months)	4
1.2.2 emergence of epistemic states (12-24 months)	4
1.2.3 “Sally-Anne” False belief task	4
1.2.4 second-order mentalizing	4
1.3 Theoretical perspectives on ToM acquisition	4
1.3.1 Nativism and core knowledge	4
1.3.2 Empiricism and connectionism/associationism	4
1.3.3 constructivism and the theory theory	4
1.4 Reconstructing constructivism	4
1.4.1 rational constructivism	4
1.4.2 the role of inductive bias	5
1.4.3 transition to chapter 2	5
2 Bayesian Model	6
2.1 Bayesian Cognitive Modeling	6
2.1.1 Reverse-engineering the mind	6
2.1.2 Marr’s levels of analysis	7
2.1.3 Rational constructivism	7
2.2 The language of thought as a representational medium	8
2.2.1 Beyond associations, to programs	8
2.2.2 λ -calculus as universal formalism	9
2.2.3 why programs for theory of mind?	9
2.2.4 why programs for theory of mind?	10
2.3 Bayesian program induction as the learning engine	11
2.4 Hierarchical Bayesian models	13
2.5 The blessing of abstraction	14
2.6 The DreamCoder algorithm	14
2.6.1 Wake: exploration/enumeration/search (Bayesian inference)	14
2.6.2 Sleep: Abstraction (library learning, compression)	15
2.6.3 Sleep: dreaming (amortized inference)	15
2.7 Learning theory of mind via inverse planning	16
2.7.1 agents as approximately rational planners	16
2.7.2 Inverting the planner	17
2.7.3 From skeletal knowledge to rich theories	18
3 Implementation	20

3.1 Overview	20
3.2 ECD architecture	20
3.2.1 Enumeration	20
3.2.2 Compression	21
4 Results and Discussion	23
4.1 Found abstractions	23
4.1.1 atomic primitives	23
4.1.2 decomposed primitives	24
4.2 Data analysis	24
4.2.1 Per-variant belief coverage + combo	28
4.2.2 Behavioral probe: Passing the false-belief test	29
4.3 Ablation experiments	31
4.3.1 Silo vs joint stitch	31
4.3.2 Corpus-mix dose-response	31
4.3.3 Budget dose-response	31
4.3.4 Dreaming ablation	31
4.3.5 Curriculum ablation	31
4.3.6 Seed robustness	31
4.4 Discussion	31
4.5 vs. real world	32
4.6 setup	32
4.7 Further work	32
4.7.1 Polymorphism	32
Bibliography	34

Chapter 1

Introduction: Theory of Mind

1.1 The nature and scope of theory of mind

The ice cream peddler's scratchy jingle got louder still as he brought his cart to a stop nearby. A child, hitherto engrossed in sandbox amusements, abandoned her playmates abruptly. She darted across the playground, hurtling at full speed towards a gaggle of parents. Absorbing the impact of a 20 kg cruise missile, the target continued his conversation unfazed by the child now tugging at his jeans. Only when he deigned to look down at him did he shriek and go pale. But the child barely had time to react, because just then she heard her name shouted from the other side of the playground. She turned to its source and ran towards a similar-looking man holding two ice cream cones.

You chuckle at the scene, observing from a nearby park bench. The girl, wanting ice cream, pestered a man she thought was her father. The man, undoubtedly accustomed to his own nagging kids, ignored the girl thinking it was his own child. Descriptions like these, in terms of desires, beliefs, and intentions, seem trivial and obvious. But there's something miraculous about them. How are you able to peer into other peoples' minds? Mental content is unobservable, all you see is a sequence of physical actions. And yet humans, even infants, are experts at inferring mental content rapidly and accurately.

Humans have the cognitive ability to impute mental states to others, to attribute a belief, desire, or intention to another person. This capacity is known as "theory of mind", "folk psychology", the ability to "mindread", to "mentalize". It is integral to everything that makes us human: to cooperate with someone you need to understand what they think, what they want. To decode someone's words, you need to begin with what they're trying to (by the Rational Speech Act framework). To understand social roles like "helping" and "hindering", you need to attribute an intention to them. Humans make these inferences so naturally that it seems like not much of an impressive ability at all, and "mindreading" seems too fabulous a term for something so mundane. But the ubiquity of these inferences should make the human capacity for mentalizing even more impressive, not less.

In this thesis we study the question: how is this capacity acquired? How do humans, even at a young age, come to acquire this ability? We'll begin by

1.1.1 the problem of other minds:

frame it as a fundamental challenge: minds are unobservable, yet humans (even infants) are experts at inferring their contents rapidly and accurately

1.1.2 the function of ToM:

discuss its role in social interaction, such as cooperation, communication (pragmatics, RSA), and understanding social roles like "helping" or "hindering"

1.2 Developmental milestones (ToM scale)

1.2.1 early roots (0-12 months)

discuss evidence that infants as young as 6 months old interpret motion as goal-directed and apply a “teleological stance”, an intuitive schema relating actions, goals, and environmental constraints (this also helps later to defend **optimize**) as non-intensional

ToM scale: Diverse desires

1.2.2 emergence of epistemic states (12-24 months)

review findings on infants’ sensitivity to perceptual access (e.g. knowing an agent can only have a goal for an object they can see)

ToM scale: Knowledge access

1.2.3 “Sally-Anne” False belief task

milestone where children transition from reality-centric responses to representing false beliefs around age 4

ToM scale: False beliefs

1.2.4 second-order mentalizing

brief overview of more complex social reasoning that emerges later (e.g. “I think that you think”), where the representations have to be richer, the model has to be higher-dimensional

1.3 Theoretical perspectives on ToM acquisition

1.3.1 Nativism and core knowledge

present the view that ToM is a built-in module, a core system [Leslie]. Discuss the “poverty of stimulus” argument, the data is too sparse for general learning. Which implies innate domain-specific constraints like the “principle of efficiency”

1.3.2 Empiricism and connectionism/associationism

contrast nativism with the view that ToM is learned through statistical patterns and bottom-up associations from experience. Challenge for associationism: the structure exists, it’s just not in the data. So how can a learner infer abstract concepts like “belief” if they aren’t in the data?

1.3.3 constructivism and the theory theory

Introduce the “child as scientist” metaphor. Argue that children don’t just associate, they construct causal models, intuitive/folk theories of the mind that they revise in light of new evidence, similar to scientific progress.

1.4 Reconstructing constructivism

1.4.1 rational constructivism

the modern bridge. It adopts the constructivist idea of theory change but seeks a more formal mechanism than classic Piagetian accounts.

1.4.2 the role of inductive bias

introduce the idea that learning “gets off the ground” by combining strong, potentially innate, domain-general mechanisms with minimal, skeletal domain-specific knowledge

could be an analyzer which highlights events resulting from one’s own actions, making the latent concept of intervention more salient. Alternatively, an innate or early-developing agency-detector might help in identifying interventions resulting from the actions of intentional agents. Altogether this suggests a novel take on nativism—a “minimal nativism”— in which strong, but domain-general, inference and representational resources are aided by weaker, domain-specific perceptual input analyzers. [Goodman 2011]

where abstract knowledge is clearly constructed, such as intuitive biology, it has been observed that the most abstract domain knowledge often comes into place first, before specific knowledge (Wellman & Gelman, 1998). The blessing of abstraction provides a potential explanation of this observation as well. Though we have argued that abstract knowledge about causality may be learnable, our results should also not be taken to support an entirely empiricist viewpoint. Our ideal learner possesses a rich language for expressing theories and a strong inductive learning mechanism. These are both significant innate structures, though ones that may be required for many learning tasks. In addition, we have shown that the domain-general mechanisms for learning and representation are greatly aided by a collection of domain-specific “perceptual input analyzers.” It may be ontogenetically cheap to build innate structures that make some intervention events salient, but quite expensive to build an innate abstract theory (or a comprehensive analyzer). Our simulations suggested that these analyzers need not be perfectly tuned to causality or cover all intervention events. There are a number of plausible candidates that have been previously suggested to support causal reasoning: animacy or agency detectors, Michottean event detectors, proprioception, etc. Since a powerful learning mechanism is present in human cognition, the most efficient route to abstract knowledge may be by bootstrapping from these simple, non-conceptual mechanisms. Thus we are suggesting a kind of minimal nativism: strong domain-general inference and representational resources, aided by weak domain-specific input analyzers. [Goodman 2011]

1.4.3 transition to chapter 2

end by noting that while the theory theory provides a powerful metaphor, it lacks a precise computational backbone to explain how children search through the infinite space of possible theories. This sets up chapter 2 to introduce bayesian program induction (via DreamCoder) as that formal implementation.

Chapter 2

Bayesian Model

2.1 Bayesian Cognitive Modeling

- Marr’s levels of analysis
- computational: what is the thing that the system is doing
- it’s doing bayesian updating. it’s finding the simplest theory.
- algorithmic: the procedure by which the solution is produced
-

identify the computational problem that the mind is trying to solve: the problem is find the best explanation. we argue that best is determined logically by bayes’ rule. so it’s doing bayesian updating. then the algorithmic level is the procedure at which it’s produced, ECD

2.1.1 Reverse-engineering the mind

Introduce the philosophy of adopting an “engineering” approach to understand human behavior by identifying the abstract computational problems that the mind must solve

This book is about how the human mind comes to understand the world—and ultimately, perhaps, how we humans may come to understand ourselves. Many disciplines, ranging from neuroscience to anthropology, share this goal—but the approach that we adopt here is quite specific. We adopt the framework of cognitive science, which aims to create such an understanding through reverse-engineering: using the mathematical and computational tools from the engineering project of creating artificial intelligence (AI) systems to better understand the operation of human thought. AI generates a rich and hugely diverse stream of hypotheses about how the human mind might work. But cognitive science does not just take AI as a source of inspiration. What we have learned about the mathematical and computational underpinnings of human cognition can also help to build more human like intelligence in machines. The fields of AI and cognitive science were born together in the late 1950s, and grew up [Griffiths 2024]

In presenting the Bayesian approach in this book, we have chosen to emphasize the fact that the underlying philosophy is one of reverse-engineering. To create a Bayesian model of cognition, we begin by thinking about the computational problem that the mind is solving. This involves identifying the data available to learn from, specifying hypotheses about how those data are being generated, and assigning prior probabilities to those hypotheses. Bayes’ rule then indicates the ideal solution to that problem, generating predictions that can be compared against human behavior. If we see a correspondence between model and behavior—and particularly if it holds up as we run further experiments designed to test the model’s predictions—then we have a way to understand why people might be doing what they are doing, and a model that we can use to make machines that perform similarly. This reverse-engineering approach instantiates a view of how to make progress in cog- [Griffiths 2024]

2.1.2 Marr's levels of analysis

bayesian models focus on the computational level, the logic of the problem, what the information processing system is doing.

[Marr 1982]

us about actual human minds. The paper ends with an appendix containing a glossary and a collection of useful resources for those interested in learning more. 2 Bayesian Basics: Inductive generalization from examples The most basic question the Bayesian framework addresses is how to update beliefs and make inferences in light of observed data. In the spirit of Marr's (1982) computational- level of analysis, it begins with understanding the logic of the inference made when generalizing from examples, rather than the algorithmic steps or specific cognitive pro- [Perfors 2011]

Lastly, the project of reverse-engineering the mind must unfold over multiple levels of analysis, only one of which has been our focus here. Marr (68) famously argued for analyses that integrate across three levels: The computational level characterizes the problem that a cognitive system solves and the principles by which its solution can be computed from the available inputs in natural environments; the algorithmic level describes the procedures executed to produce this solution and the representations or data structures over which the algorithms operate; and the implementation level specifies how these algorithms and data structures are instantiated in the circuits of a brain or machine. Many early Bayesian models addressed only the computational level, characterizing cognition in purely functional terms as approximately optimal statistical inference in a given environment, without reference to how the computations are carried out (25, 39, 69). The HBM of learning and development discussed here target a view between the computational and algorithmic levels: cognition as approximately optimal inference in probabilistic models defined over a learner's subjective and dynamically growing mental representations of the world's structure, rather than some objective and fixed world statistics. [Tenenbaum 2011]

Marr (1982) argued that information-processing systems can be analyzed at three levels: the computational level characterizes the problem that a system solves and the principles by which its solution can be computed from the available inputs in natural environments; algorithmic-level analysis describes the procedures executed to produce this solution and the representations or data structures over which the algorithms operate; and the implementa- tion level specifies how these algorithms and data structures are instantiated in the circuits of a brain or machine. Reverse-engineering means beginning at the computational level, trying to understand the function of a system before diving into algorithms and implementation. For this reason, we have referred to it as a top-down or function-first approach (Griffiths et al., 2010). This idea was made explicit in Anderson's (1990) framework for rational analysis, which focuses on analyzing cognition in terms of adaptive solutions to prob- lems posed by the environment, and resulted in several groundbreaking Bayesian models of cognition. Many early Bayesian models addressed only the computational level, characterizing cog- [Griffiths 2024]

2.1.3 Rational constructivism

position the framework as a middle way between nativism and empiricism, where domain-general statistical mechanisms construct domain-specific structural knowledge

associations). Many developmental researchers rejected this choice altogether and pursued less formal approaches to describing the growing minds of children, under the headings of “constructivism” or the “theory theory” (Gopnik & Meltzoff, 1997). The potential to explain how people can genuinely learn with abstract structured knowledge may be the most salient feature of Bayesian cognitive models—the biggest reason for their popularity in some developmental circles (Gopnik & Tenenbaum, 2007; Griffiths, Chater, Kemp, Perfors, & Tenenbaum, 2010; Perfors, Tenenbaum, & Wonnacott, 2010; Griffiths, Sobel, Tenenbaum, & Gopnik, 2011b; Xu, 2019; Spelke, 2022), and the biggest target of skepticism from others, in both the traditional nativist and empiricist camps (Berwick, Pietroski, Yankama, & Chomsky, 2011; McClelland et al., 2010). [Griffiths 2024]

chology that similarly grew out of a desire to answer the tension between representation and learning in a boundedly rational system. On one side, a purely nativist response was to deny learning and to focus on characterizing the detailed representations that were already in place. On another side, an empiricist response was to suggest that structured representations were not necessary, that learning (and the inferences that followed) was simply a bottom-up process of learning statistical associations. Other debates played out in cognitive development as well. Some research tended to characterized children as “noisy” or “irrational” adults, while other research sought to demonstrate that children are efficient and effective rational learners. With the development of the probabilistic framework came a renewed interest in unifying [Griffiths 2024]

Most importantly, the Bayesian approach lets us move beyond classic either-or dichotomies that have long shaped and limited debates in cognitive science: “empiricism versus nativism,” “domain-general versus domain-specific,” “logic versus probability,” “symbols versus statistics.” Instead we can ask harder questions of reverseengineering, with answers potentially rich enough to help us build more humanlike AI systems. How can domain-general mechanisms of learning and representation build domain-specific systems of knowledge? How can structured symbolic knowledge be acquired through statistical learning? The answers emerging suggest new ways to think about the development of a cognitive system. Powerful abstractions can be learned surprisingly quickly, together with or prior to learning the more concrete knowledge they constrain. Structured symbolic representations need not be rigid, static, hard-wired, or brittle. Embedded in a probabilistic framework, they can grow dynamically and robustly in response to the sparse, noisy data of experience. [Tenenbaum 2011]

2.2 The language of thought as a representational medium

2.2.1 Beyond associations, to programs

contrast the symbolic approach (structured, compositional) with the connectionist approach (unstructured, associative weights), arguing that programs capture the productivity and open-endedness of human thought

defined over large numerical vectors. Learning was seen as estimating strengths in an associative memory, weights in a neural network or parameters of a high-dimensional, nonlinear function (McClelland & Rumelhart, 1986; Rogers & McClelland, 2004). Bayesian cognitive models, in contrast, have had the most success with defining probabilities over more structured symbolic forms of knowledge representations used in computer science and AI, such as graphs, grammars, predicate logic, relational schemas, and functional programs. Different forms of representation are

used to capture people’s knowledge in different domains and tasks, as well as at different levels of abstraction. In learning words and concepts from examples, the knowledge that guides both children [Griffiths 2024]

The Form of Abstract Knowledge Abstract knowledge provides essential constraints for learning, but in what form? This is just question 2. For complex cognitive tasks such as concept learning or causal reasoning, it is impossible to simply list every logically possible hypothesis along with its prior and likelihood. Some more sophisticated forms of knowledge representation must underlie the probabilistic generative models needed for Bayesian cognition. In traditional associative or connectionist approaches, statistical models of learning were defined over large numerical vectors. Learning was seen as estimating strengths in an associative memory, weights in a neural network, or parameters of a high-dimensional nonlinear function (12, 14). Bayesian cognitive models, in contrast, have had most success defining probabilities over more structured symbolic forms of knowledge representations used in computer science and artificial intelligence, such as graphs, grammars, predicate logic, relational schemas, and functional programs. Different forms of representation are used to capture people’s knowledge in different domains and tasks and at different levels of abstraction. [Tenenbaum 2011]

operates over structured (Fodor, 1975; Fodor & Pylyshyn, 1988), probabilistic (Lake et al., 2017; Goodman et al., 2015), program-like (Turing, 1936; Baum, 2004) representations. While there have been many other proposals for modeling conceptual representations, only programs arguably capture the full breadth and depth of people’s algorithmic abilities (Goodman et al., 2015). Code can model both procedural and declarative information and allow them to interact seamlessly. Universal programming languages integrate all this knowledge into a single formal representation. [Rule 2020]

2.2.2 λ -calculus as universal formalism

explain why λ -calculus is a natural choice for modeling concepts as stochastic functions that can be combined and recombined

2.2.3 why programs for theory of mind?

We begin by defining the stochastic λ -calculus, a mathematical system that is able to represent complex generative models using only a few basic constructs. It is based on the λ -calculus, which is one of the principal models of deterministic computation. After introducing this mathematical basis, we will add a number of features to make modeling easier, making this a more practical probabilistic programming language (PPL) for cognitive science. In particular, we will adopt the PPL Church (Goodman et al., 2012), which elegantly extends the sparse mathematical system of stochastic λ -calculus based on the Scheme dialect of Lisp. How should we begin to build a compositional formal system—a language—for express- [Griffiths 2024]

some fundamental knowledge that is more than just associations between words and cardinalities. We present a formal learning model which shows that statistical inference over a sufficiently powerful representational space can explain why children follow this developmental trajectory. The model uses several pieces of machinery, each of which has been independently proposed to explain cognitive phenomena in other domains. The representational system we use is lambda calculus, a formal language for compositional semantics (e.g., Heim & Kratzer, 1998; Steedman, 2000), computation more generally (Church, 1936), and other natural-language learning tasks (Piantadosi, Goodman,

Ellis, & Tenenbaum, 2008; Liang, Jordan, & Klein, 2009, 2010; Zettlemoyer & Collins, 2005, 2007). The core inductive part of the model uses Bayesian statistics to formalize what inferences learners should make from data. This involves two key parts: a likelihood function which measures how well hypotheses fit observed data, and a prior which measures the complexity of individual hypotheses. We use simple and previously proposed forms of both. The model uses a likelihood function that uses the size principle (Tenenbaum, 1999) to penalize hypotheses which make overly broad predictions. Frank, Goodman, and Tenenbaum (2007) proposed that this type of likelihood function is important in cross-situational word learning and Piantadosi et al. (2008) showed that it could solve the subset problem in learning compositional semantics. The prior is from the rational rules model of Goodman, Tenenbaum, Feldman, and Griffiths (2008), which first linked probabilistic inference with formal, compositional, representations. The prior assumes that learners prefer simplicity and re-use in compositional hypotheses and has been shown to be important in accounting for human rule-based concept learning. [Piantadosi 2012]

come to have meaning as CRS supposes, by virtue of how the structures they are mapped to act on other symbols. Learners are able to derive new facts by applying their internal expressions to each other in novel ways. As I show, this can give rise to rich systems of knowledge that span classes of computations and permit learners to extend a few simple observations into the domain of richer cognitive theories. 2 Combinatory Logic as a Language for Universal Isomorphism A mathematical system known as combinatory logic provides the formal tool we'll use to construct a universal isomorphism language as a hypothesized LOT. Combinatory logic was developed in the early- and mid-1900s in order to allow logicians to work with expressions that did not require variables like "x" and "y", yet had the same expressive power (Hindley and Seldin 1986). Combinatory logic's usefulness is demonstrated by the fact that it was invented at least three independent times by mathematicians, including Moses Schönfinkel, John vonNeumann, and Haskell Curry (Cardone and Hindley 2006). The main advantages of combinatory logic are its simplicity (allowing us to posit very minimal built-in machinery) and its power (allowing us to model symbols, structures, and relations). In cognitive research, combinatory logic is primarily seen in formal theories of natural language semantics (Steedman [Piantadosi 2019])

2.2.4 why programs for theory of mind?

social reasoning requires recursive, compositional representations to handle nested mental states. There's no cleaner way of doing this than with programs. it's basically already a program.

The pattern of using an embedded query to capture the choices of another agent is a very general pattern for modeling intuitive psychology (Stuhlmüller & Goodman, 2013). We could write down the abstract structure schematically as: (define choice (lambda (belief state goal?) (query (define action (action-prior)) action (goal? (belief state action)))))) where belief is taken to be the agent's summary of the world dynamics (transitions from states to states, given actions), and goal? is a goal predicate on states picking out those that the agent desires. Of course many additional refinements and additions may be needed to build an adequate model of human intuitive psychology—agents form [Goodman 2015]

Probabilistic Programs as a Unifying Language of Thought 467 probabilistic programming for Bayesian inverse planning to more abstract symbolic planning settings, using the Gen PPL

(Cusumano-Towner et al., 2019) and the Planning Domain Description Language to represent an agent’s world models, goals, and plans using Gen also enables explicit probabilistic modeling of a boundedly rational agent’s approximate planning algorithm to make goal inference robust when the agent might be prone to planning mistakes of various kinds. Of course, many additional components will be needed to build fully adequate models of [Griffiths 2024]

or the blocked agent is previously associated as negative, or...’ One alternative to a cue-based account is to use generative models of action choice, as in the Bayesian inverse planning (or “Bayesian theory-of-mind”) models of Baker, Saxe, and Tenenbaum (2009) or the “naive utility calculus” models of Jara-Ettinger, Gweon, Tenenbaum, and Schulz (2015) (See also Jern and Kemp (2015) and Tauber and Steyvers (2011), and a related alternative based on predictive coding from Kilner, Friston, and Frith (2007)). These models formalize explicitly mentalistic concepts such as ‘goal,’ ‘agent,’ ‘planning,’ ‘cost,’ ‘efficiency,’ and ‘belief,’ used to describe core psychological reasoning in infancy. They assume adults and children treat agents as approximately rational planners who choose the most efficient means to their goals. Planning computations may be formalized as solutions to Markov Decision Processes (or POMDPs), taking as input utility and belief functions defined over an agent’s state-space and the agent’s state-action transition functions, and returning a series of actions the agent should perform to most efficiently fulfill their goals (or maximize their utility). By simulating these planning processes, people can predict what agents might do next, or use inverse reasoning from observing a series of actions to infer the utilities and beliefs of agents in a scene. This is directly analogous to how simulation engines can be used for intuitive physics, to predict what will happen next in a scene or to infer objects’ dynamical properties from how they move. It yields similarly flexible reasoning abilities: Utilities and beliefs can be adjusted to take into account how agents might act for a wide range of novel goals and situations. Importantly, unlike in intuitive physics, simulation-based reasoning in intuitive psychology can be nested recursively to understand social interactions – we can think about agents thinking about other agents. [Lake 2016]

2.3 Bayesian program induction as the learning engine

Prior vs. Likelihood: Introduce Bayes’ Rule as the engine of learning, where the learner trades off the simplicity of a program (the prior) against its fit to observed behavior (the likelihood)

compute this probability, $P(h/d) \propto P(h)P(d/h)$, (1) identifying the posterior probability as proportional to the product of the prior and the likelihood. This Bayesian posterior provides a rational analysis of inductive learning: a coherent integration of evidence and a priori knowledge into posterior beliefs (optimal within the specified learning context). By viewing concept acquisition as an inductive problem, we may employ this Bayesian framework to describe the learning of a concept from examples. As described earlier, our hypothesis space is the collection of all phrases in a grammatically-generated concept language. Since each concept of this language is a classification rule, a natural likelihood is given by simply evaluating this rule on the examples: the data have zero probability if they disagree with the classification rule, and constant probability otherwise. However, to account for the unreliability of examples, we will allow non-zero probability for a data set, even if some examples are misclassified. That is, we assume that there is a small probability that any example is an outlier, which should be ignored. We will see in the next section that this outlier assumption combines with

rule-evaluation to give a simple likelihood function, which decreases exponentially with the number of “misclassifications” of the examples. [Goodman 2008]

Bayesian inference gives a rational framework for updating beliefs about latent variables in generative models given observed data (Jaynes, 2003; Mackay, 2003). Background knowledge is encoded through a constrained space of hypotheses H about possible values for the latent variables—candidate world structures that could explain the observed data. Finer-grained knowledge comes in the prior probabilities $P(h)$ that specify the learner’s degree of belief in each hypothesis h prior to (or independent of) the observations. Bayes’ rule updates these prior probabilities to posterior probabilities $P(h/d)$ conditional on the observed data d : [Griffiths 2024]

natural interpretations in many problems. The likelihood reflects the fit between hypothesis and data, and the prior indicates the a priori plausibility of a hypothesis (which might decrease for hypotheses that have low frequency, are complicated, or seem otherwise improbable). The contribution of these two factors to the conclusions that we should draw is fairly natural and makes intuitive sense in a variety of contexts. Returning to an example introduced in chapter 1, if you see John coughing (your data d), you might consider three hypotheses about the cause of the cough: a cold (h_1), lung disease (h_2), or heartburn (h_3). You might rule out heartburn on the basis of fit, since it might only slightly increase the chance of coughing. A cold and lung disease both fit well with the cough—they increase the probability of coughing—but they differ in plausibility. Normally, a cold is far more common than lung disease, and thus might be the hypothesis that you would select to explain the coughing. However, the plausibility of these two hypotheses might change if you were passing by a hospital and saw John coughing inside. All inductive inferences require considering fit and plausibility. Bayes’ rule just tells you how they should be combined to reach a conclusion, using the common language of probability theory to determine the impact of each factor. In the context of cognitive science, priors become a useful way of describing the inductive [Griffiths 2004]

Summary of the basic modeling framework. Although both priors and likelihoods can be understood on their own terms, it is only in combination that they explain how people can successfully learn the extensions of new words from just a few positive examples. Successful word learning requires both a constrained space of candidate hypotheses—provided by the prior—and the ability to reweight hypotheses according to how well they explain a set of observed examples—provided by the likelihood. Without the constraints imposed by the prior, no meaningful generalizations would be possible. Without the likelihood, nothing could be learned from multiple examples beyond simply eliminating inconsistent hypotheses. In particular, priors and likelihoods each contribute directly to the main pattern of generalization that we described in the introduction and that we look for in our experiments: Given just a single example of a novel kind label, generalization to other objects should be graded, but given several examples, learners should apply the word more discriminately, generalizing to all and only members of the most specific natural concept that spans the observed examples. The prior determines which concepts count as “natural,” whereas the likelihood generates the specificity preference and determines how the strength of that preference—and thus the sharpness of generalization—increases as a function of the number of examples. [Xu 2007]

2.4 Hierarchical Bayesian models

Explain how learning overhypotheses enables a learner to acquire the *form* of a theory (i.e. agents are rational) before learning specific goals

The formalism of hierarchical Bayesian modeling makes it possible to express the assumptions relating knowledge at multiple levels of abstraction (Gelman, Carlin, Stern, & Rubin, 1995), and Bayesian inference over such a model describes an ideal learner of abstract knowledge (Tenenbaum, Griffiths, & Kemp, 2006). Though real learning is undoubtedly resource-constrained, the dynamics of an ideal learner can uncover unexpected properties of what it is possible to learn from a given set of evidence. For instance, it has been reported (e.g. Kemp, Perfors, & Tenenbaum, 2007) that learning at the abstract level of a hierarchical Bayesian model is often surprisingly fast in relation to learning at the more specific levels. We term this effect the blessing of abstraction¹: abstract learning in an hierarchical Bayesian model is often achieved before learning in the specific systems it relies upon, and, as a result, a learner who is simultaneously learning abstract and specific knowledge is almost as efficient as a learner with an innate (i.e. fixed) and correct abstract theory. Hierarchical Bayesian models have been used before to study domain-specific abstract causal knowledge (Kemp, Goodman, & Tenenbaum, 2007), and simple relational theories (Kemp et al., 2008). Here we combine these approaches to study knowledge of causality at the most abstract, domain general level. [Goodman 2011]

trees, causal networks, and other forms of structure that people appear to know explicitly (Rogers & McClelland, 2004). Bayesian cognitive modelers have answered these challenges by combining the structured knowledge representations described in this chapter with advanced methods from Bayesian statistics known as hierarchical Bayesian models (HBMs; Gelman, Carlin, Stern, & Rubin, 1995). HBMs address the origins of hypothesis spaces and priors by positing not just a single level of hypotheses to explain the data, but multiple levels: hypothesis spaces of hypothesis spaces, with priors on priors. Bayesian inference across all levels allows hypotheses and priors needed for a specific learning task to themselves be learned at larger or longer-time scales, at the same time as they constrain lower-level learning (see chapter 8). In machine learning and AI, HBMs have primarily been used for transfer learning or [Griffiths 2024]

This paper suggests that hierarchical Bayesian models (Good, 1980; Gelman, Carlin, Stern & Rubin, 2003) can help to explain the computational principles which allow overhypotheses to be learned. Hierarchical Bayesian models (HBMs) include representations at multiple levels of abstraction, and show how knowledge can be acquired at levels quite remote from the data given by experience. To illustrate these points, we describe one of the simplest possible HBMs and use it to suggest how overhypotheses about feature-variability (e.g. the shape bias) are acquired and used to support categorization. We also present an extension of this basic model that groups categories into ontological kinds (e.g. objects and substances) and discovers the features and the patterns of feature variability that are characteristic of each kind. [Kemp 2009]

schema discovers the disease-symptom framework theory by assigning variables 1 to 6 to class C1, variables 7 to 16 to class C2, and a prior favoring only $C1 \rightarrow C2$ links. These assignments, along with the effective number of classes (here, two), are inferred automatically via the Bayesian Occam's razor. Although this three-level model has many more degrees of freedom than the model

in (B), learning is faster and more accurate. With $n = 80$ patients, the causal network is identified near perfectly. Even $n = 20$ patients are sufficient to learn the high-level $C1 \rightarrow C2$ schema and thereby to limit uncertainty at the network level to just the question of which diseases cause which symptoms. (D) A HBM for learning an abstract theory of causality (62). At the highest level are laws expressed in first-order logic representing the abstract properties of causal relationships, the role of exogenous interventions in defining the direction of causality, and features that may mark an event as an exogenous intervention. These laws place constraints on possible directed graphical models at the level below, which in turn are used to explain patterns of observed events over variables. Given observed events from several different causal systems, each encoded in a distinct data matrix, and a hypothesis space of possible laws at the highest level, the model converges quickly on a correct theory of intervention-based causality and uses that theory to constrain inferences about the specific causal networks underlying the different systems at the level below. [Tenenbaum 2011]

2.5 The blessing of abstraction

abstract knowledge can be acquired faster than specific facts, by pooling evidence across multiple scenarios. This could explain how children acquire core social constraints early in life.

The formalism of hierarchical Bayesian modeling makes it possible to express the assumptions relating knowledge at multiple levels of abstraction (Gelman, Carlin, Stern, & Ru-bin, 1995), and Bayesian inference over such a model describes an ideal learner of abstract knowledge (Tenenbaum, Griffiths, & Kemp, 2006). Though real learning is undoubtedly resource-constrained, the dynamics of an ideal learner can uncover unexpected properties of what it is possible to learn from a given set of evidence. For instance, it has been reported (e.g. Kemp, Perfors, & Tenenbaum, 2007) that learning at the abstract level of a hierarchical Bayesian model is often surprisingly fast in relation to learning at the more specific levels. We term this effect the blessing of abstraction¹: abstract learning in an hierarchical Bayesian model is often achieved before learning in the specific systems it relies upon, and, as a result, a learner who is simultaneously learning abstract and specific knowledge is almost as efficient as a learner with an innate (i.e. fixed) and correct abstract theory. Hierarchical Bayesian models have been used before to study domain-specific abstract causal knowledge (Kemp, Goodman, & Tenenbaum, 2007), and simple relational theories (Kemp et al., 2008). Here we combine these approaches to study knowledge of causality at the most abstract, domain general level. [Goodman 2011]

Blessing of abstraction : The phenomenon whereby higher-level, more abstract knowl- [Perfors 2011]

2.6 The DreamCoder algorithm

2.6.1 Wake: exploration/enumeration/search (Bayesian inference)

The wake phase solves the inverse problem: searching for the program (intention) that most likely generated the data (actions)

[Ellis 2020]

D are unknown. The marginal likelihood of the observed tasks is then given by $p_{\theta D}(X) = \prod_{x \in X} \sum_{\rho} p(x | \rho) p_{\theta D}(\rho)$, where $p(x | \rho)$ is the likelihood of x being produced, and hence solved, by

ρ . To learn a good generative model that maximises the likelihood, we need to know which programs score highly under $p(x \mid \rho)$ —i.e. solve our tasks. We have seen previously why this is challenging: discovering programs that can account for the observed tasks requires search. To help with search, a recognition (inference) model $q\phi D (\rho \mid x)$ is learnt to infer the programs that are most likely to solve a given task. The recognition model parameters ϕD map tasks to distributions over programs that, as with the generative model, specifies the probability that components part of the library D are used. Estimating ϕD is done using both (ρ, x) pairs sampled from the generative model (fantasies) and programs found to solve the observed tasks $x \in X$ (replays). The recognition model is used to search for programs that solve a task x by enumerating programs in decreasing order of their probability under $q\phi D (\rho \mid x)$. Programs that solve x are stored in a task-specific set Bx . [Palmarini 2024]

2.6.2 Sleep: Abstraction (library learning, compression)

Explain how refactoring and compression automate the discovery of new social primitives, implementing the Theory Theory by allowing the learner to grow their own language of thought

Discovering programs that may have produced the observed tasks (those in $\{Bx\}_{x \in X}$) now provides more data to infer the parameters ϑD generating them. Inferring ϑD entails choosing the library D whose components they control. Rather than maximise the likelihood directly, DREAM-CODER performs maximum a posteriori (MAP) inference using a prior over libraries D and parameters ϑD . Maximising the MAP objective (which can only be approximated) w.r.t. D corresponds to updating D to include functions that best compress the discovered solutions. After updating D , parameters ϑD are updated to their MAP estimates. [Palmarini 2024]

its descendants (Wong, Ellis, Tenenbaum, & Andreas, 2022; Bowers et al., 2023) are recent AI approaches for Bayesian program learning inspired by Church and the PLoT hypothesis that show how such “library learning” can dramatically expand the repertoire of effectively learnable concepts, given a limited amount of computational resources. These systems work not only by adding newly learned concepts to the library, but also by abstracting out program components that are implicitly shared between previously learned concepts to identify the most explanatory and compact theory of a domain—effectively performing a heuristic version of hierarchical Bayesian inference to learn the prior for generating novel concepts. LAPS (Language for Abstraction and Program Search) (Wong et al., 2022) extends DreamCoder to learn a joint prior on programs and natural-language translations of those programs, and models how—consistent with studies in cognitive development (Carey, 2009)—experience with natural language in a domain can bootstrap a learner’s theory acquisition and joint acquisition of word meanings and novel concepts, well beyond what could be gleaned merely from observed perceptual examples. [Griffiths 2024]

2.6.3 Sleep: dreaming (amortized inference)

explain the neural recognition network as a way to “compile” slow, deliberative Bayesian search into fast, “intuitive” social reasoning.

2.7 Learning theory of mind via inverse planning

2.7.1 agents as approximately rational planners

defining the generative model. agents move in an environment (MDP/POMDP) to maximize utility efficiency

One alternative to a cue-based account is to use generative models of action choice, as in the Bayesian inverse planning (or “Bayesian theory-of-mind”) models of Baker, Saxe, and Tenenbaum (2009) or the “naive utility calculus” models of Jara-Ettinger, Gweon, Tenenbaum, and Schulz (2015) (See also Jern and Kemp (2015) and Tauber and Steyvers (2011), and a related alternative based on predictive coding from Kilner, Friston, and Frith (2007)). These models formalize explicitly mental-istic concepts such as ‘goal,’ ‘agent,’ ‘planning,’ ‘cost,’ ‘efficiency,’ and ‘belief,’ used to describe core psychological reasoning in infancy. They assume adults and children treat agents as approximately rational planners who choose the most efficient means to their goals. Planning computations may be formalized as solutions to Markov Decision Processes (or POMDPs), taking as input utility and belief functions defined over an agent’s state-space and the agent’s state-action transition functions, and returning a series of actions the agent should perform to most efficiently fulfill their goals (or maximize their utility). By simulating these planning processes, people can predict what agents might do next, or use inverse reasoning from observing a series of actions to infer the utilities and beliefs of agents in a scene. This is directly analogous to how simulation engines can be used for intuitive physics, to predict what will happen next in a scene or to infer objects’ dynamical properties from how they move. It yields similarly flexible reasoning abilities: Utilities and beliefs can be adjusted to take into account how agents might act for a wide range of novel goals and situations. Importantly, unlike in intuitive physics, simulation-based reasoning in intuitive psychology can be nested recursively to understand social interactions – we can think about agents thinking about other agents. [Lake 2016]

*Humans are adept at inferring the mental states underlying other agents’ actions, such as goals, beliefs, desires, emotions and other thoughts. We propose a computational framework based on Bayesian inverse planning for modeling human action understanding. The framework represents an intuitive theory of intentional agents’ behavior based on the principle of rationality: the expectation that agents will plan approximately rationally to achieve their goals, given their beliefs about the world. The mental states that caused an agent’s behavior are inferred by inverting this model of rational planning using Bayesian inference, integrating the likelihood of the observed actions with the prior over mental states. This approach formalizes in precise probabilistic terms the essence of previous qualitative approaches to action understanding based on an “intentional stance” [Dennett, D. C. (1987). *The intentional stance*. Cambridge, MA: MIT Press] or a “teleological stance” [Gergely, G., Nádasdy, Z., Csibra, G., & Biró, S. (1995). Taking the intentional stance at 12 months of age. *Cognition*, 56, 165–193]. In three psychophysical experiments using animated stimuli of agents moving in simple mazes, we assess how well different inverse planning models based on different goal priors can predict human goal inferences. The results provide quantitative evidence for an approximately rational inference mechanism in human goal inference within our simplified stimulus paradigm, and for the flexible nature of goal representations that human observers can adopt. We discuss the implications of our experimental results for human action understanding in real-world contexts, and suggest how our framework might be extended to capture other kinds of mental state*

inferences, such as inferences about beliefs, or inferring whether an entity is an intentional agent. [Baker 2009]

analog of expected utility theory used for understanding everyday behavior. Importantly, this does not assume that humans necessarily plan or act rationally in all circumstances. Rather, we tacitly expect others to behave rationally (or approximately so) in particular situations, and perform “explanation by rationalization” by attributing the mental states which make their behavior appear most rational within the present context. The Bayesian Theory of Mind (BToM) framework combines these principles governing the [Baker 2012]

To address these questions, we formalize action understanding as a Bayesian inference problem. We model the intuitive causal relation between beliefs, goals and actions as rational probabilistic planning in Markov decision problems (MDPs), and invert this relation using Bayes’ rule to infer agents’ beliefs and goals from their actions. We test our framework with psychophysical experiments in a simple setting that allows us to collect a large amount of fine-grained human judgments to compare with the strong quantitative predictions of our models. [Baker 2012]

Bayesian Models of Cognitive Development 513 *Skeletal structured generative models are one route for embodying and implementing core knowledge, but other approaches are being developed as the AI and machine learning community re-engages with findings from cognitive development. One such promising direction, specifically in intuitive physics, is to combine artificial neural networks or graphs with different minimal notions of objects, and to let the network discover the dynamics and interactions between objects (for several recent examples, see Mrowca et al., 2018; Battaglia et al., 2016, 2018; Chang, Ullman, Torralba, & Tenenbaum, 2016). A very different computational approach, however, is try to recover the principles of core knowledge purely from vast amounts of empirical data (for recent examples in intuitive physics and psychology, respectively, see Piloto et al., 2018; Rabinowitz et al., 2018). Such blank-slate models on their own do not yet generalize well, but it is too early to say [Griffiths 2024]*

2.7.2 Inverting the planner

Show how the observer uses Bayesian induction to work backward from a trajectory to the most likely goals and beliefs

analog of expected utility theory used for understanding everyday behavior. Importantly, this does not assume that humans necessarily plan or act rationally in all circumstances. Rather, we tacitly expect others to behave rationally (or approximately so) in particular situations, and perform “explanation by rationalization” by attributing the mental states which make their behavior appear most rational within the present context. The Bayesian Theory of Mind (BToM) framework combines these principles governing the [Baker 2012]

To address these questions, we formalize action understanding as a Bayesian inference problem. We model the intuitive causal relation between beliefs, goals and actions as rational probabilistic planning in Markov decision problems (MDPs), and invert this relation using Bayes’ rule to infer agents’ beliefs and goals from their actions. We test our framework with psychophysical experiments in a simple setting that allows us to collect a large amount of fine-grained human judgments to compare with the strong quantitative predictions of our models. [Baker 2012]

eral Bayesian treatment of “theory of mind” or “mentalizing”: the core human capacity to observe other agents’ actions and infer the beliefs, desires, and intentions that gave rise to them, assuming that those actions were the result of approximately rational goal-directed planning and decision-making processes in the agent’s mind. Chapter 14 develops the Bayesian approach to modeling theory of mind with a focus on understanding the actions of agents moving around us and in our local environment, on relatively short spatial and temporal scales, as developed in seminal work by Baker, Saxe, and Tenenbaum (2009), Lucas, Griffiths, et al. (2014), Jern, Lucas, and Kemp (2017), Jara-Ettinger, Gweon, Schulz, and Tenenbaum (2016), Baker, Jara-Ettinger, Saxe, and Tenenbaum (2017), and Ullman et al. (2009). At its core, this framework posits that individuals infer the intentions and beliefs of others by observing their actions and then “inverting” the agents’ planning process. Instead of predicting future actions based on given intentions (as in forward planning and rational decision-making; see chapter 7), Bayesian inverse planning starts with the observed actions and works backwards to infer the most likely intentions and beliefs that could have produced those actions. To achieve this, the model uses Bayes’ rule, combining prior beliefs about the actor’s preferences and intentions with the likelihood of the observed actions, given that those intentions assume an approximately rationally planner, to produce a posterior distribution over possible intentions. This method provides a structured way to interpret the behavior of others in terms of underlying mental states, giving us not only a candidate mechanism behind the human ability to understand and predict the actions of others, but also opening up a vast landscape of insights and modeling opportunities for human social cognition and the cultural basis of cognition more generally. Finally, the Bayesian approach can be used to shed light on some aspects of cultural [Griffiths 2024]

tion (14.24)). Using the resulting policy as the generative model, joint belief (the probability distribution over states), desire (the latent reward function), and competence (the underlying cost function) can be inferred through Bayesian inference, where given an observed trajectory t , $p(B, R, C|t) = p(t|B, R, C)p(B)p(R)p(C)$. (14.25) Baker et al. (2017) developed and tested the model that we have just presented, asking adult participants to make joint inferences about agents’ beliefs and desires based on how they navigated an environment. Figure 14.11 shows several scenarios from this experiment, in which a hungry graduate student leaves their office to walk to lunch at one of three food trucks: Korean (K), Lebanese (L), or Mexican (M). There are two parking spots for the trucks (marked in yellow), and trucks can park in different spots on different days, or not show up at all, so the student may not know where each truck is parked and must plan carefully where to walk to get lunch from the best truck available as quickly as possible. Using a POMDP for a generative model, the agent’s desires can be captured using a reward function that represents their preferences over trucks, and the agent’s initial beliefs can be represented as a probability distribution over each of three partially observable world states: the Northeast parking spot being occupied by (1) Lebanese (L) or (2) Mexican (M), or (3) being empty (N for none). Finally, observations of the trucks are determined by line of sight, with a small probability of observation failure. Consider figure 14.11c, in which the student can initially see the Korean truck in the [Griffith 2024]

2.7.3 From skeletal knowledge to rich theories

show how DreamCoder can start with a skeletal planner (the principle of efficiency) and iteratively construct a library of complex social concepts like “helping”, “hindering”, or “chasing”

Skeletal structured generative models are one route for embodying and implementing core knowledge, but other approaches are being developed as the AI and machine learning community re-engages with findings from cognitive development. One such promising direction, specifically in intuitive physics, is to combine artificial neural networks or graphs with different minimal notions of objects, and to let the network discover the dynamics and interactions between objects (for several recent examples, see Mrowca et al., 2018; Battaglia et al., 2016, 2018; Chang, Ullman, Torralba, & Tenenbaum, 2016). A very different computational approach, however, is try to recover the principles of core knowledge purely from vast amounts of empirical data (for recent examples in intuitive physics and psychology, respectively, see Piloto et al., 2018; Rabinowitz et al., 2018). Such blank-slate models on their own do not yet generalize well, but it is too early to say [Griffiths 2024]

C.L. Baker et al. / Cognition 113 (2009) 329–349 Kautz, H., & Allen, J. (1986). Generalized plan recognition. In Proceedings of the fifth national conference on artificial intelligence (pp. 32–37). Kemp, C., Perfors, A., & Tenenbaum, J. B. (2007). Learning overhypotheses with hierarchical Bayesian models. Developmental Science, 10(3), 307–321. Körding, K. (1997). Decision theory: What “should” the nervous system do? Science, 318(5850), 606–610. Liao, L., Fox, D., & Kautz, H. (2004). Learning and inferring transportation routines. In Proceedings of the nineteenth national conference on artificial intelligence (pp. 348–353). [Baker 2009]

programming languages—as candidate mechanistic models for thinking. And finally, by drawing on hierarchical Bayesian frameworks for learning inductive constraints and learning to learn, the PLoT lets us think in terms of programs as hypotheses for novel concepts, and probabilistic meta-programs that generate domain-specific languages of programs as powerful and dynamically evolvable hypothesis spaces and priors for inductive concept learning. Looking ahead, we should start by acknowledging that Church models, and PLoT models [Griffiths 2024]

Each cycle the system is additionally tested on the domain’s held-out tasks. Testing time is consistent across all domains: the system is provided 10 minutes per task to search for a solution using its current library and recognition model. Fig. 5A (Row 1) shows the percentage of test tasks solved in each cycle by all systems. Except for text editing, where performance remains comparable across all systems, utilizing the recognition model for chunking (dream decompiling) enables faster domain proficiency and enhanced generalization through the learnt library. This distinction is most evident during the intermediate cycles of learning, following similar performance in the initial iterations and before proceeding to converge again in the later iterations. Notably, this occurs despite all systems having solved a similar number of training tasks throughout (Fig. 6, Appendix C). At the respective peak differences (excluding text editing), DDC-PC outperforms DREAMCODER by 13.25% on average test [Palmarini 2024]

Chapter 3

Implementation

3.1 Overview

Modularity-nativists argue for the existence of a Theory of Mind Module (ToMM), a natively-endowed domain-specific system for understanding the behavior of intentional agents. In Alan Leslie’s characterization the ToMM introduces attitude concepts like *Believes* and uses a “proprietary representational system” for ascribing attitudes to agents [Les94]. Our goal is to demonstrate that the ability to theorize in mental terms can in principle be *learned*, without the need for a native ToMM.

In our first trial we show that, given a non-mental representational substrate, a system can learn a functional abstraction that encodes a notion of belief attribution. The learned function `fork(derive, commit)`

We initialize the system with a library of domain-general primitives, none of which posit an agent’s intentional state, nor encodes attitude concepts like *believes*. Lacking bespoke ToM capacities, such a system would on Leslie’s view not be able to reason about intentional agents, to attribute a belief to an agent, to explain an agent’s movement as navigation according to a private intensional representation. We show that the system learns *useful abstractions* of domain-general functions, abstractions that encode attitude concepts like *Believes* and attribute an intensional state to an agent.

In the second iteration, we show that the implementation can arrive at an abstracted notion of belief starting from an even more basic language of 2-ary combinators.

3.2 ECD architecture

ECD is a wake-sleep library-learning loop inspired by the DreamCoder [Ell+20]. Each iteration of the loop consists of an enumeration phase, a compression phase, and a dreaming phase. In enumeration, the system generates candidate programs to solve tasks

3.2.1 Enumeration

Candidate programs are enumerated in order from most to least probable, i.e. from lowest to highest cost. To illustrate, say we’re searching for a program that solves a task depicting the value 3 rising in each successive grid, so our target program is `(step 3 up)`. Its head is `step :: cellvalue, direction -> fn` and its leaves are `3 :: cellvalue` and `up :: direction`. A program’s cost is given by the sum of the costs of its constituent primitives. Say Q is the initial flat prior, the type-uniform distribution, so each primitive’s cost is the negative natural log of the number of symbols in the grammar that share its type. For example, there are $N = 4$ symbols of type `direction` so each gets $p = \frac{1}{N} = \frac{1}{4}$. By Shannon, we have $\text{cost}(\text{up}) = -\ln p = -\ln \frac{1}{4} = \ln 4 \approx 1.386$ nats. By the same process, `step`’s cost is $\ln 8 \approx 2.1$ nats and `3`’s cost is $\ln 10 \approx 2.3$. By logarithmic additivity, to find the program’s total cost we simply sum the costs of its constituent primitives which yields ≈ 5.8 nats for `(step 3 up)`.

Program search proceeds by budget window to ensure that cheaper programs are enumerated first. For (**step 3 up**), the first two windows are skipped by pruning. For example in window [2, 4) after paying 2.1 nats for **step**, only 1.9 nats remain to fill the two holes in the partial program tree. **step**’s tail types (arguments) are **cellvalue** and **step**. But any primitive of type **cellvalue** costs 2.3 nats, so **step**’s first argument can’t be placed. So trees with a **step** node are pruned, since they would never complete within the second budget window. So search is cheapest-first, since more complex programs only become affordable once the window is wide enough.

At budget window [4, 6) the ceiling is finally sufficiently wide to admit all three symbols. The tree is assembled incrementally: each enumerated subtree is appended to a copy of its parent node, and a callback fills the remaining argument slots by the same recursion (so programs of any arity or nesting depth are handled by the same procedure). Since enumeration is type-directed, each argument slot draws candidates only from symbols of the matching type, so no ill-typed partial tree is ever constructed. (**step 3 up**) completes at its total cost of ≈ 5.8 nats. Since the tree has no more available holes, the callback fires: the finished program is run on the task’s input grids and its output is compared against the target. Since it’s the first solution found, it is stored as the task’s program. Because the windows are tried in increasing order of cost, the first solution the search returns is (up to ties within a window) the shortest program that solves the task — its minimum-description-length explanation under the current grammar.

3.2.2 Compression

Enumeration minimizes the description length of each program *under a fixed grammar*. But the grammar it uses is itself a choice: in the first round it is just the flat type-uniform prior, which prices (**step 3 up**) at ≈ 5.8 nats whether that shape of program is used by one task or by fifty. Compression is the step that lets the system revise the grammar in light of what enumeration actually found. It asks a different question — not “what is the shortest program for *this* task?” but “what single change to the library makes the *whole corpus of solutions* shortest?” This is the minimum-description-length objective one level up: we minimize the joint cost of the library plus the programs written in it, $DL(\text{library}) + \sum_{\text{tasks}} DL(\text{program} \mid \text{library})$, rather than the cost of any one program alone.

The trade-off is what makes the objective non-trivial. Consider a fragment — a subtree, possibly with holes — that recurs across many solutions. Naming it as a new library primitive **fn_k** costs something: every subsequent use of that name is one token rather than the several it abbreviates, but the library is now larger, and since a symbol’s cost is $-\ln$ of the number of symbols sharing its type, adding one more **fn**-typed symbol raises the cost of *every* **fn** token by a small amount. Against that grammar cost stands the benefit: each occurrence of the fragment collapses from a multi-node subtree to a single leaf. The abstraction is a net win exactly when the fragment is large enough, and recurs often enough, that the tokens saved across the corpus outweigh the price of carrying one more symbol. So an accidental one-off is never abstracted, while a shared structural motif — precisely the thing we hope belief is — pays for itself.

We delegate the search for these abstractions to **stitch** [Bow+23], a top-down library-learning system (`saturate_stitch`, `ecd.py:285`). Where the DreamCoder line searches version spaces bottom-up, **stitch** enumerates candidate abstractions top-down and uses a compressive-utility bound to find, without materializing every rewrite, the fragment whose abstraction most reduces the joint

description length. One call runs for a fixed number of *iterations*: each iteration commits the single most compressive abstraction, rewrites the corpus in terms of it, and repeats, so later abstractions may be built out of earlier ones; `max_arity` bounds how many holes (`#0`, `#1`, ...) an abstraction may carry. The programs we hand it are first *normalized* — every previously-invented abstraction is inlined back to bare primitives — so that stitch sees only the primitive alphabet and its discoveries never collide with names from an earlier round. What it returns is a set of abstractions (fragments with `#i` holes) together with the corpus *rewritten* to use them; we remap each `#i` hole to a `$i` argument, infer the hole’s type, and register the abstraction as a new `fn_k` in the library `D`.

The crux of the design is that this compression is *joint*. Enumeration must proceed per root type — one budget walk produces one kind of arrow, so the trajectory tasks (`fn`) and the registration tasks (`fn_p_g`) are searched separately — but every solution they find, across both root types and across both the mind-attributing and the minds-free families, is pooled into a *single* stitch call. There is one library and one compression objective, not one silo per family. This is what makes an abstraction for belief an honest MDL win rather than an artefact of searching the belief tasks in isolation: the same objective that could have paid for belief’s structure is free to spend its budget on the overlay, registration, and obstacle families instead, and a fragment is abstracted only if it earns its keep against all of them at once.

What the pooled stitch reliably discovers is the research payload of the whole system. Across the phases it invents a single agent constructor, in Phase 1 of the form

```
fn_0 = (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
         (sync_to_world $0))
```

in which the agent value `$0` appears *twice* — once in the policy that plans over the private, phantom-walled grid built by `derive`, and again in the `sync_to_world` commit that writes the result back to the world. Nothing in the grammar forced those two argument slots to be filled by the same value; that they collapse into one shared hole is the structural signature of agency — an agent acting on its own model of the world — and it is *discovered* by the description-length objective, not stipulated by any primitive. The rewritten corpus stitch returns then becomes the training data for the dreaming phase and the starting library for the next round, in which programs a level deeper — belief compounds too expensive to enumerate from bare primitives — have become reachable because their shared core now costs a single token.

Chapter 4

Results and Discussion

belief/theory-of-mind is a discoverable compound under MDL rather than a built-in primitive

4.1 Found abstractions

4.1.1 atomic primitives

round 1 enumeration solves obstacle tasks

```
[ 1037] caught (compose (optimize (neg_dist 5) 4) (wall_at c1 c0))
```

and simple a few (7) simple false belief tasks

```
[102603] caught (fork (compose (wall_at c1 c2) (optimize (neg_dist 1) 7)) (sync_to_world 7))
```

so at the end of the round it abstracts a function for placing a wall and navigating around it

```
fn_0: (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) [cellvalue, cellvalue, coord, coord] -> fn
```

and uses `fn_0` to abstract the belief signature `fork(derive, commit)`: on the model grid, run `fn_0` as the `derive`, then run `sync_to_world` as the `commit` using the same `#0` (agent value) as passed to `fn_0`. So run the calculation for `#0` based on the modified grid, and then commit the conclusion to the real world.

```
fn_3: (fork (fn_0 #0 #3 #2 #1) (sync_to_world #0)) [cellvalue, coord, coord, cellvalue] -> fn
```

round 2 enumeration solves the rest of the simple false belief tasks using `fn_3`, requiring just around 1% of the original enumeration cost. So it can successfully use the learned abstraction to generalize. It's much easier to solve the false belief tasks when it already has a function and just needs to fill in the values, rather than needing to rediscover the entire structure of belief again

```
[ 1053] caught (fn_3 4 c3 c2 5)
```

at the end of the round it abstracts the belief signature again, which was `fn_3` in round 1

```
fn_6: (fork (compose (wall_at #3 #2) (optimize (neg_dist #1) #0)) (sync_to_world #0)) [cellvalue, cellvalue, coord, coord] -> fn
```

and it abstracts simple desire

```
fn_7: (optimize (neg_dist #0)) [cellvalue, cellvalue] -> fn
```

so then in **round 3** it can use those to solve witness-belief tasks

```
[ 11634] caught (compose (fn_6 5 8 c3 c3) (fn_7 2 1))
```

final abstractions: belief constructor

```
fn_6 [cellvalue, cellvalue, coord, coord] -> fn
body: (fork (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)) (sync_to_world $0))
```

4.1.2 decomposed primitives

final abstractions

```
fn_6 [cellvalue, cellvalue, coord, coord] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at $3 $2) (optimize (neg_dist $1) $0)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief – degenerate scope-complement commit) ***

fn_7 [cellvalue, cellvalue] -> fn
body: (optimize (neg_dist $0) $1)
  (desire fragment)

fn_8 [cellvalue, cellvalue, coord, coord] -> fn
body: (compose (wall_at $3 $2) (optimize (neg_dist $1) $0))
  (obstacle/belief policy: stamp wall ▶ navigate – the shared derive)

fn_9 [coord, coord, cellvalue, cellvalue] -> fn
body: (compose (optimize (neg_dist $3) $2) (wall_at $1 $0))
  (obstacle/belief policy: stamp wall ▶ navigate – the shared derive)

fn_10 [coord, cellvalue, cellvalue] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd (compose (wall_at c2 $0) (optimize (neg_dist $1) $2)))) sync_all)
  *** AGENT TYPE CONSTRUCTOR (belief – degenerate scope-complement commit) ***

fn_11 [fn, fn_p_g] -> fn
body: (pipe_gpg (compose_gp dup (mapsnd $0)) $1)
```

4.2 Data analysis

The guiding motivation of this project is that an intensional theory, attributing a belief to an agent, is the most compressed representation of an agent’s behavior. This is obvious, since the agent navigates based on its intensional representation, so a theory that accounts for this will better represent the underlying data generating process. It’s certainly possible to describe an agent’s movement without attributing a belief to it, but it would be a laborious explanation that accounts for each step in the sequence. And of course it doesn’t generalize well, it overfits to a specific agent’s movement in a particular scene.

To illustrate this we show that no non-intensional solution is as short as one which stipulates belief attribution. As described in chapter 3 the enumerator processes candidate programs in $-\log p$ order and saves just the first (cheapest) solution it finds, call it *found*. For false belief tasks should be a program that attributes a belief to an agent. Since a false belief task can also be solved by

a purely extensional program, call any such program a *rival* since it’s an extensional rival to the intensional explanation.

For each rival, we generate a *margin* score by taking $\text{margin} = \text{DL}(\text{rival}) - \text{DL}(\text{found})$, where DL is the description length (how many nats it takes to write program p under library L). So for any rival, if $\text{margin} > 0$ then the intensional program is shorter than the extensional, otherwise an extensional program is at least as short. We plot the empirical cumulative distribution over the margin values for all rivals, so the x -axis shows possible margin values and the y -axis shows the cumulative fraction of rival programs with $\text{margin} \leq x$. Since description length depends on the given library we plot two ECDFs: dotted for programs written in the base library, and solid for programs written in the final library (enriched with abstractions found through five ECD rounds).

We first give the plot for the higher level DSL in which `fork` and `sync` are given as atomic primitives.

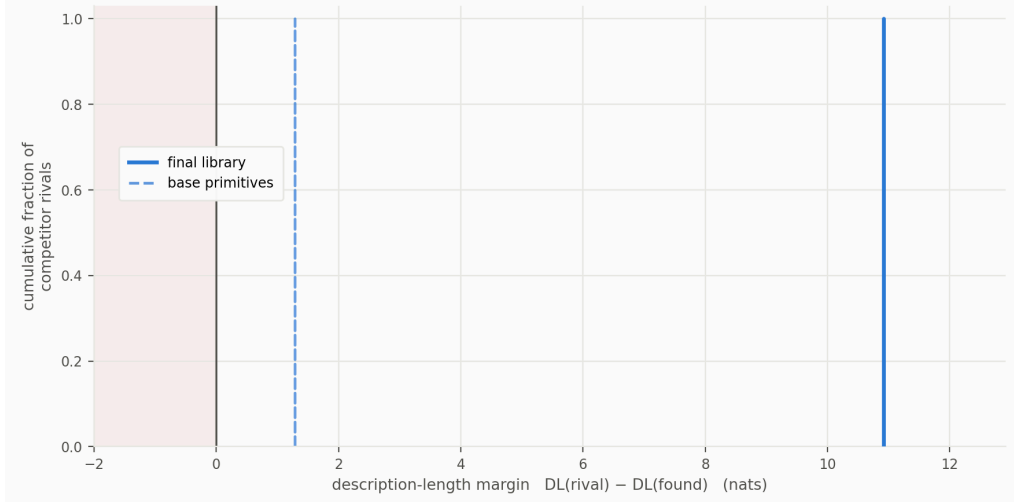


Figure 1: ECDF of MDL margin over rival programs (atomic DSL)

Both curves sit entirely to the right of $x = 0$, so for each of the 24 false belief tasks, the *found* (intensional) program is a shorter description length than any *rival* (extensional) program. In the base (dashed) library, rival programs cost only ≈ 1.3 nats more than the found program. But the margin balloons to ≈ 10.9 in the final (solid) library: once the `fork(derive, commit)` abstraction is added, the enumerator can immediately generate programs that posit an agent’s intensional state. Without such a learned abstraction, an intensional program is only slightly less verbose than an extensional one. But with the abstraction, intensional programs are vastly more compressed than extensional ones.

This effect is even more pronounced in the plot for the lower-level DSL, in which `fork` and `sync` are decomposed to their product category components.

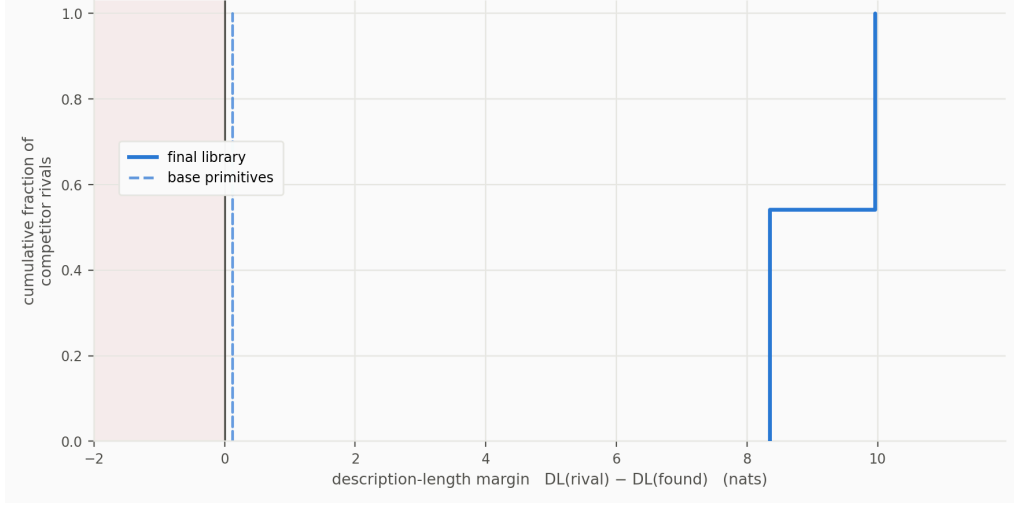


Figure 2: ECDF of MDL margin over rival programs (decomposed DSL)

Under the base primitive library, the mental reading is barely shorter than the physical one (≈ 0.1 nats). Since the intensional machinery of forking and syncing needs to be constructed from lower-level components, length of intensional programs end up virtually tied with that of the verbose extensional programs. But once the belief abstraction `fn_0` is learned and added to the library, the margin grows to $\approx +8.4$ nats. So mental programs go from being just as bad as non-mental ones to being vastly better, through learning a fitting abstraction. Note that the jump to a $\approx +9.96$ nats margin for the last 11 tasks is due to a further specialized `fn_4 = (fn_0 ... c2)` that hard-codes a phantom-wall at a particular coordinate `c2` so such programs are one coordinate cheaper. But the rival cost is constant accross both, so the entire step is a property of how tightly the library compressed the intensional side.

We can also see this effect when comparing description lengths of different families of tasks: belief, desire, physics, and world. To plot, we fix each corpus (task type) and calculate each corpus’s description length given a round as such:

$$\text{DL}(C|r) = \sum_{c \in C} \text{DL}(p_c|L_r) + \text{DL}(L_r)$$

where C is a task corpus (e.g. belief tasks), c is an instance of a task in that corpus, r is an ECD round, and p_c is the found (minimal) program that solves task c . So calculate the description length of a corpus at a given round, for each task in the corpus $c \in C$ we sum the DL of each program p_c that solves it, given under that round’s library L_r . And we add the description length of the library L_r itself, a structure term to account for the cost of a more complicated library.

Under the base library, total description length (black line) of the entire task corpus (every C) is 3209 nats. After four ECD rounds it drops to 2076 nats, an 1134 nat compression. We see the biggest drop in the belief corpus, once the system learns to efficiently represent intentional agents using the `fork(derive, commit)` abstraction. Once that constructor is in the library, every belief task rewrites through it and the corpus gets dramatically shorter at once.

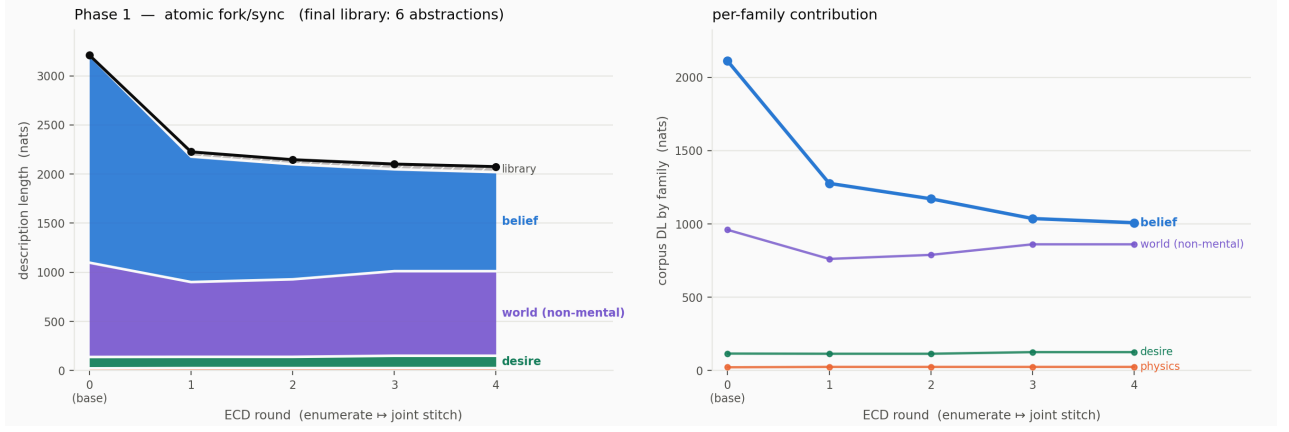


Figure 3: Description length per task family (atomic primitives)

Unlike the belief corpus, the description length of the other (non-mental) tasks remains mostly stable as new abstractions are learned. They’re solved just as easily under the base primitives as they are under the final library (a bit worse actually since the library is more complex). If the initial primitives *did* encode mental notions, if there were a native ToMM, we’d see similar results for the belief tasks: they’d be solved just as easily from the start. But since iterated abstraction yields a significant improvement, it’s clear that the initial primitives are not mental but that the ECD loop enriches the library with abstractions that *do* encode mental concepts (the concept of belief attribution).

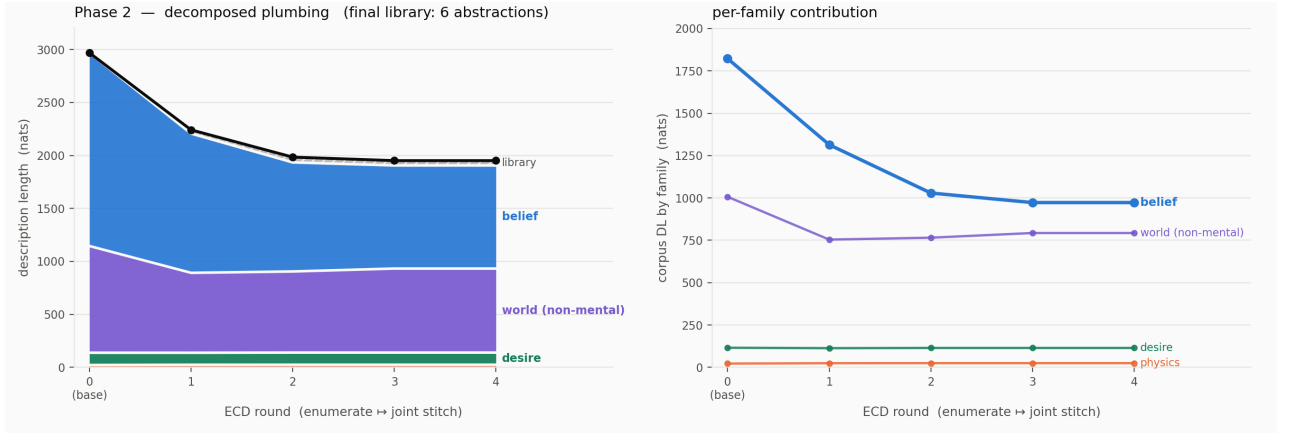


Figure 4: Description length per task family (decomposed primitives)

Belief tasks are reached late and cheaply. They only become solvable once the ECD loop has abstracted the reusable parts, and the moment that abstraction is added to the library, belief tasks flip from timing out to being solved in seconds. See the following plots of cumulative task solutions per round, and per-task solve time.

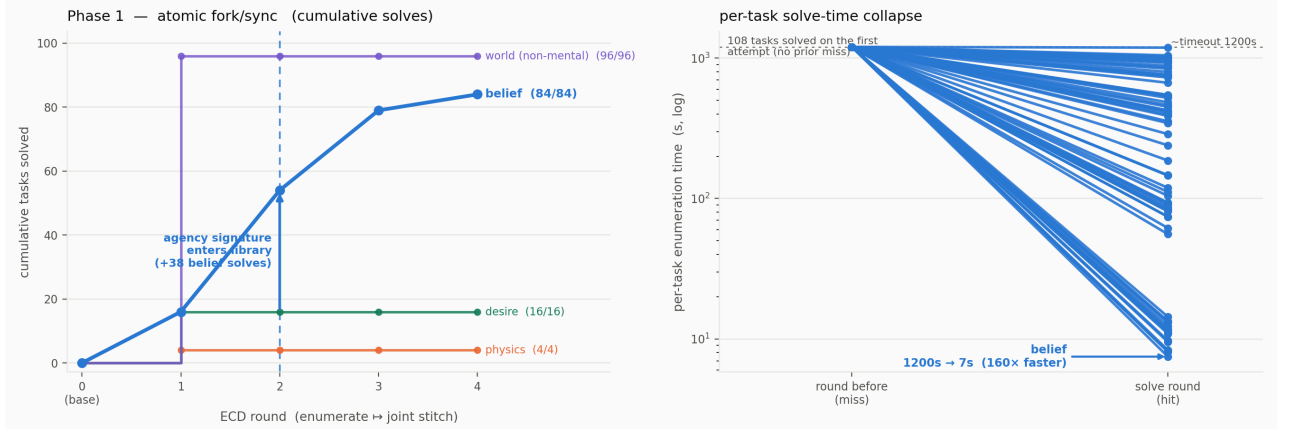


Figure 5: Cumulative solves by task family (atomic primitives)

Tasks other than belief get solved in the first round, they don't need abstractions. But belief can't be enumerated from base primitives, so it exhibits an S-curve. There are very few solves early on, since each solution must be laboriously composed of deep nested primitives. Then there's a sharp step up when the stitch finally abstracts the agency signature, the shared-model structure belief reuses. Then the learned abstraction is used to solve the remaining belief tasks, so the solved tasks plateau for the last round.

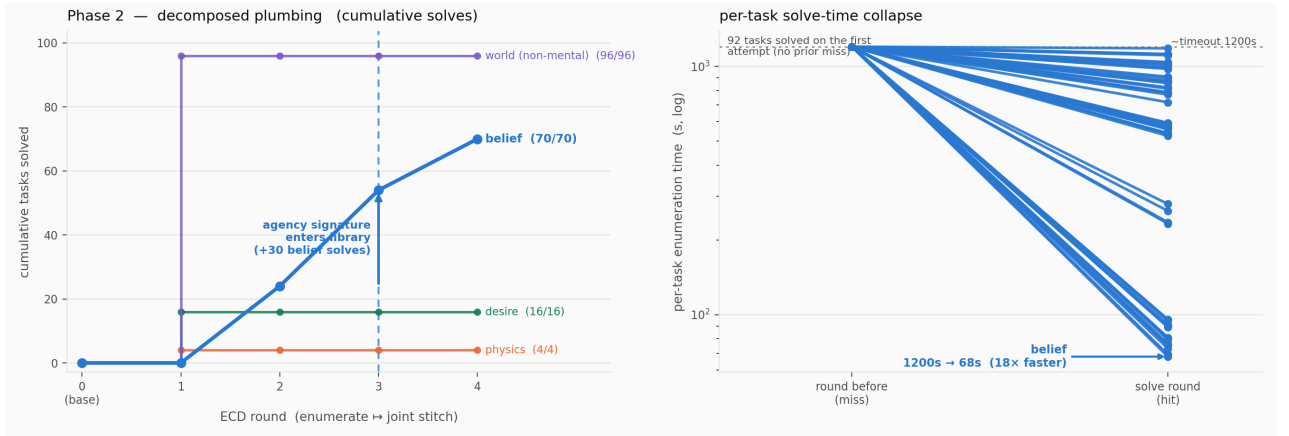


Figure 6: Cumulative solves by task family (decomposed primitives)

Starting with the atomic library, the abstraction jump occurs at round 2 where we abruptly see +38 belief solves. If we start with the decomposed library, the abstraction takes an extra round to assemble so the step is at round 3 where we see +30 belief solves.

Plot 2: per-task solve time, round n vs round $n+1$, as a paired slope-graph or log-scale scatter — the 1100 s $\rightarrow$ 10 s collapse is currently buried in log text.

4.2.1 Per-variant belief coverage + combo

Plot 1: Bar chart of solved/total for wall/witness/goal/dual

Plot 2: heatmap of $(gv, av) \times$ abstraction-used showing fn_8 tiling every cell.

Rows are the (goal value, agent value) combinations in a task, columns are the invented abstractions added to the library. The count indicates how many solutions use that abstraction.

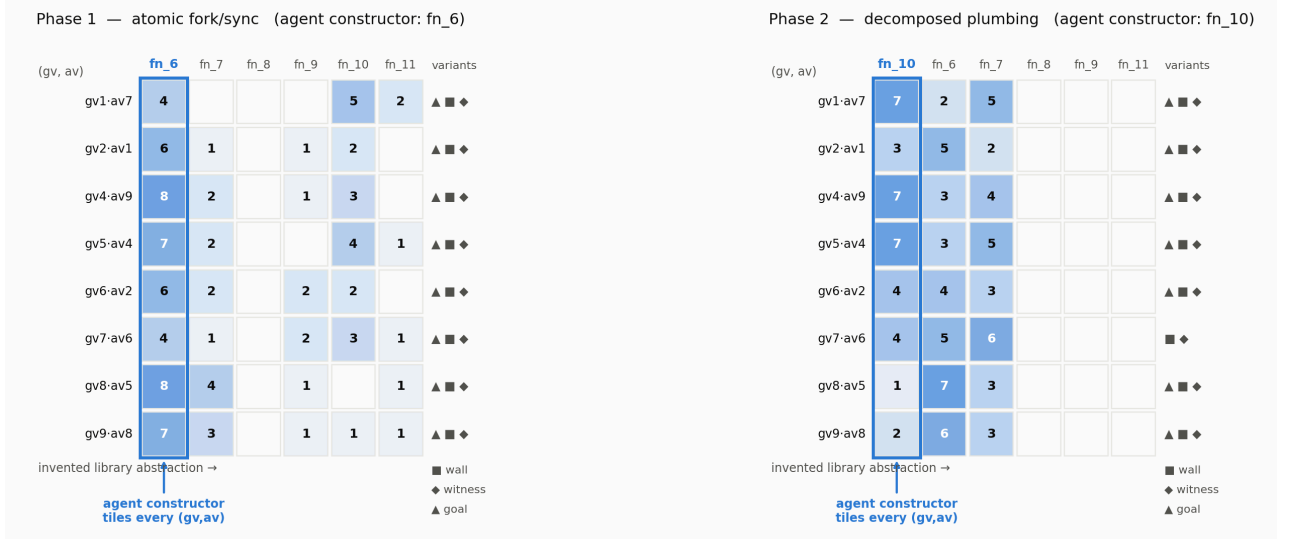


Figure 7: Usage of each abstraction across instances of belief tasks

So the belief abstraction generalizes across the (goal, agent) combinations it was trained on.

4.2.2 Behavioral probe: Passing the false-belief test

The analyses so far are about description length. They show that the intensional program is the shorter account of the data, and the abstraction the loop learns is what makes it short.

We now

But there is a more direct question we can ask of the learned library, the one the developmental literature actually asks of a child. Put the learner in front of a novel false-belief scene and see *where it predicts the agent will look*. This is the classic false-belief setup [WP], of which the Sally-Anne task is the familiar form: an object is where the agent last saw it, but the agent’s belief about its location is now false; a child who attributes belief points to the *believed* location, while a younger child who cannot yet attribute belief points to the object’s *true* location.

We reuse the goal-displacement family for this, because unlike the wall and witness families it lets the two readings come apart *behaviorally*. In a wall-belief scene the transient-wall rival reproduces the entire trajectory (that is exactly why the MDL margin, not behavior, is what separates them there). But in a goal-displacement scene the agent walks to where it believes the goal is — one cell from the true goal, which never moves — so a program that does not represent the agent’s belief cannot reproduce the walk. The two readings predict the agent settling on *different cells*. That divergence is the false-belief test.

The probe reuses the phase-1 run and re-stitches its solutions to rebuild exactly the library it converged to, the same reconstruction the MDL-margin experiment performs. Nothing is re-enumerated. Onto a *held-out* scene — drawn from a fresh seed that never entered the run, and checked to be absent from the training corpus — we instantiate the discovered belief compound (the `fork(derive, commit)` abstraction the library reprices belief through) and, as its foil, the shortest program expressible in the same library that does not represent belief. Figure 8 shows one such scene.

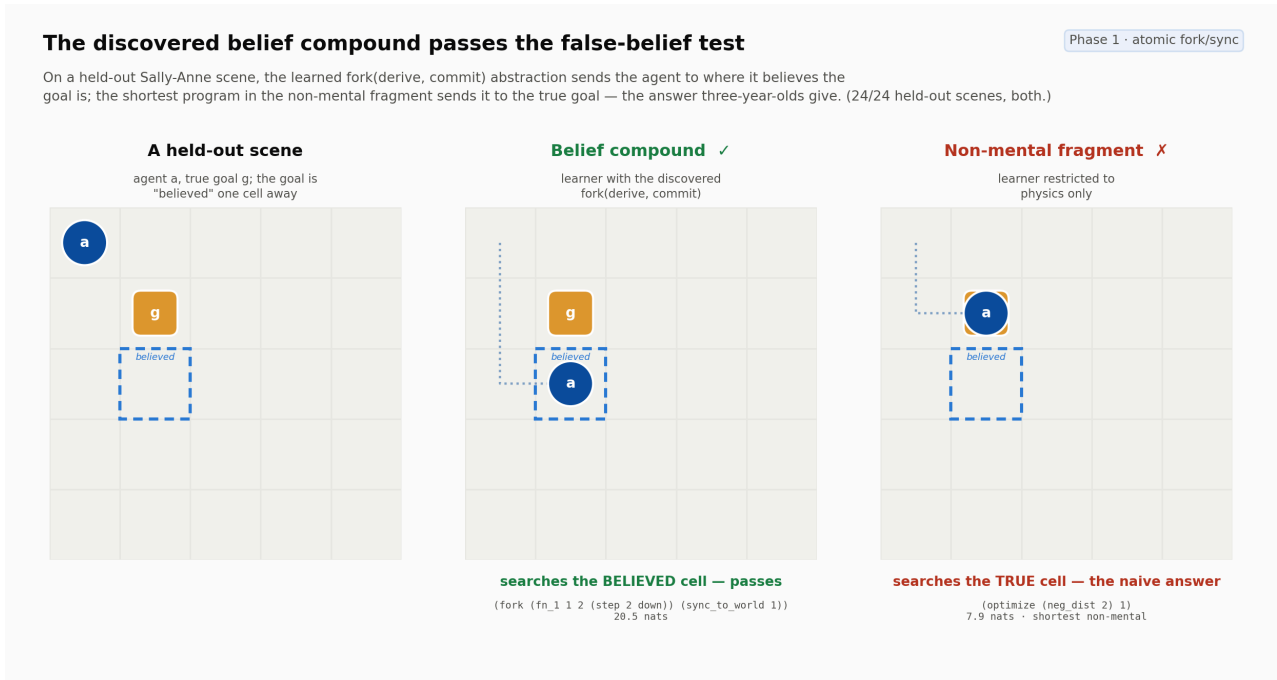


Figure 8: Behavioral probe on a held-out Sally-Anne scene. The learned belief compound sends the agent to the believed cell (passes); the shortest non-mental program under the same library sends it to the true goal (the naive answer).

The learner equipped with the belief compound writes `(fork (fn_1 1 2 (step 2 down)) (sync_to_world 1))` — the discovered `fork(derive, commit)` abstraction applied to a derive that privately displaces the goal and seeks it, committing only the agent’s own move back to the world via `sync_to_world` — and its final frame puts the agent on the *believed* cell, one step past the true goal. It searches where the agent thinks the goal is. The learner restricted to the non-mental fragment writes the shortest thing that fragment can say, plain desire `(optimize (neg_dist 2) 1)`, and its agent walks straight to the *true* goal. It searches where the goal really is. Across all 24 held-out scenes the split is clean: the belief compound lands on the believed cell every time, the shortest non-mental program on the true cell every time.

Note that here the non-mental program is *shorter* than the intensional one (7.9 nats against 20.5), which is the opposite of the MDL-margin result — and this is the point. The two figures are complementary. Where the transient-wall rival reproduces the scene, it does so only at a longer description length, so MDL rules it out; where the pure-desire rival is cheaper, it does so only by predicting the wrong behavior, so the data rule it out. There is no non-mental program that is both as short and as accurate as the belief compound. The naive reading is available and even parsimonious, exactly as it is for a three-year-old, but it fails the test. A learner that has compressed its experience into a belief compound passes the false-belief task; a learner confined to the non-mental fragment fails it in the specific way the developmental literature documents — predicting the true location rather than the believed one. The abstraction the loop discovers for the sake of compression turns out to be the very thing that licenses the mentalistic prediction.

4.3 Ablation experiments

4.3.1 Silo vs joint stitch

compress belief solutions alone vs pooled with the non-mental corpus. Is the constructor still found, and at what DL? Quantifies the file-16 claim that belief is an MDL win, not a silo artefact.

Plot: library-DL and constructor-invention bar pairs.

4.3.2 Corpus-mix dose-response

vary the non-mental fraction (0%, 25%, 50%, ...); measure constructor invention and belief solve rate. Shows the non-mental tasks are load-bearing for the discovery, i.e. domain-generalty is doing work.

Plot: solve rate / invention frequency vs mix.

4.3.3 Budget dose-response

–t-fn at 600/1200/2400 s; belief solve rate per variant. Supports “the 45 misses are budget-bound, not representational.”

Plot: solve rate vs budget, per variant (dual should be the steepest riser).

4.3.4 Dreaming ablation

–no-dream vs dreamed Q: cumulative solves vs wall-clock. Round 3-vs-4 in your log already hints the dreamed Q matters; make it a curve.

4.3.5 Curriculum ablation

with/without the 24 scaffold tasks: rounds-to-constructor and final belief coverage. Preempts “you seeded the abstraction.”

4.3.6 Seed robustness

5 corpus seeds $\times$ the main run; report every verdict metric as $k/5$ with solve-rate error bars. Cheap on the cluster, transforms every claim from anecdote to statistic.

4.4 Discussion

out of the full $2 \times 2 \times \dots$ cube of channel-direction / channel-target / z-order / scope variants, joint MDL still selects exactly the read-model $\rightarrow$ write-world, single-av corner for belief, and leaves the symmetric variants attached to their non-mental tasks.

pair apparatus was available to every task, but recruited by only the false belief ones. Those are the trajectories that can’t be accounted for by only a single grid, so the search pays + 10 nats for the second grid only when nothing cheaper explains the data. Representationalist’s criterion: posit the hidden world only if it pays for itself, only if its necessary. Availability is not use. The inference “this agent needs a belief” is made, per task, by cost-driven model selection.

The (grid, grid) and arrow types aren’t smuggled-in ToM, they’re just the logical form that ToM requires. Intensionality is the coexistence of world and model. Any substrate expressive enough to even state “act on the world via a transformed model of it” must be able to hold two representations at once. And “two things held at once” is a product, in some guise (a pair, two registers, two tape regions, two variables). You cannot discover ToM in a substrate that can’t hold two representations [for the same reason you can’t discover addition in a language with no notion of “two numbers.” The

pair is to theory of mind what the integers are to arithmetic.] And the pair is symmetric and content-free: `dup` makes (w, w) . There’s nothing in it that says “the second one is the model, the believed world.” The asymmetry that makes one copy real and the other believed is: transform the second (`mapsnd`, not `mapfst`), commit from model to world (`sync_to_world`). This asymmetry is entirely in the composition, which is the discovered part. The medium is neutral, it could go any way, as seen in the other tasks which do use those other primitives and wire them in other ways. But the system is able to discover this particular composition of these particular primitives to solve belief tasks, via programs that constitute a theory of mind.

Sure you can keep going. Decompose `dup` from a flat memory/tape with `allocate/read/write`, so “hold a second world-model” becomes a discovered pattern of buffer use. But be clear-eyed: that relocates the regress, it doesn’t end it. “Allocate a buffer” is then the primitive, and it’s no more or less mental than “form a pair” — the capacity-for-plurality is presupposed by intensionality at every level, because it’s constitutive of intensionality. There is no substrate that discovers ToM “from nothing”; there’s only substrates whose primitives are, or aren’t, individually non-mental and general.

generality by reuse is what shows it

1. the same combinators used to create the belief abstraction are used in non-mental tasks
 2. the library is also initialized with symmetric primitives to the ones used in the belief abstraction
- so it’s a domain-general library that the search happened to recruit for belief.

4.5 vs. real world

obviously this isn’t how an infant learns theory of mind

I’m missing the *self*. I’m focusing on learning to distinguish between different entities in the domain, but not an infant learning to distinguish others from itself. For example, if I bite myself it hurts, but if I bite someone else it doesn’t. maybe pain occurs only if I bite this but not that (extensional). or we have different states, different experiences, and sometimes someone else can be experiencing pain even when I’m not.

I’m missing the scale. I’m focusing on a narrow timescale. in real infants this would unfold over a long time scale.

4.6 setup

to what extent is the ToM already present in the input data and DSL and structure

4.7 Further work

4.7.1 Polymorphism

I’m using a simply-typed monomorphic type system, used only to prune enumeration. product-category composers. No generic composer. They constrain which wirings are legal, not what the program must prove. So I could further do dependent types, types as search constraint. Our product-category typing (`dup/mapsnd/compose_gp`, the `fn_p_g` discipline) is monomorphic. Dependent types are the limit case where the type carries enough information to make the I/O examples redundant. Then the MDL-over-examples approach is what you do when you can’t or won’t encode

the spec as a type, when the spec is a behavior to be discovered, as it is for a cognitive concept like belief. Belief is precisely the kind of concept you can't hand-write a dependent type for, since it would be so complicated. So it makes more sense to just have it be discovered.

DreamCoder uses polymorphic request types, so unification can rule out ill-typed branches early. But since our DSL has only ~ 30 primitives our bottleneck isn't branching factor, so monomorphic dict lookup is sufficient.

The product-category combinator family in our type system is

```
dup_g      : grid -> pair_gg
mapsnd     : fn   -> fn_p_p
compose_gp : fn_g_p, fn_p_p -> fn_g_p
pipe_gpg   : fn_g_p, fn_p_g -> fn
```

which are just the pair-grounded instantiation of universal categorical combinators

- `dup` : `a -> pair a a` (the diagonal Δ)
- `mapsnd` : `(b -> c) -> pair a b -> pair a c` (the bifunctor 'second')
- `compose` : `(b -> c) -> (a -> b) -> (a -> c)`

so if we had polymorphism we could collapse the above family into one `compose`.

With monomorphic types, a stitch abstraction discovered at type `grid` can only be reused at `grid`. With polymorphism, an abstraction abstracted over $\forall a$ can be reused across instantiations of any type a . Phase 3 hand-engineers this with `eh cons/nil` stack leaving arity as a free parameter. If we had polymorphism it would just fall out of the type discipline.

The decomposition is polymorphic in principle. Our monomorphic primitives are the ground instances at `a = grid`. So belief *can* be assembled from standard categorical combinators.

To do something like that, we'd replace bare string types with a small ADT:

- `ground ('con', 'grid', [])`
- `variable ('var', 'a')`
- `applied ('con', 'pair', [t1, t2])`
- `arrows (args, ret)`

We'd implement Robinson unification

Bibliography

- [Les94] Leslie AM. Pretending and believing: issues in the theory of ToMM. *Cognition* 1994;50:211–38. [https://doi.org/10.1016/0010-0277\(94\)90029-9](https://doi.org/10.1016/0010-0277(94)90029-9).
- [Ell+20] Ellis K, Wong C, Nye M, Sable-Meyer M, Cary L, Morales L, et al. DreamCoder: Growing generalizable, interpretable knowledge with wake-sleep Bayesian program learning 2020. <https://doi.org/10.48550/arXiv.2006.08381>.
- [Bow+23] Bowers M, Olausson TX, Wong L, Grand G, Tenenbaum JB, Ellis K, et al. Top-Down Synthesis for Library Learning. *Proceedings of the ACM on Programming Languages* 2023;7:1182–213. <https://doi.org/10.1145/3571234>.
- [WP] Wimmer H, Perner J. Beliefs about beliefs: Representation and constraining function of wrong beliefs in young children's understanding of deception n.d.